

National University of Computing and Emerging Sciences				
	Course:	Data Structures	Course Code:	CS 2001
	Program:	BS (CS)	Semester:	Fall 2025
	Name:		Section:	BCS-3D
	Roll No.		Quiz-1 (A)	

Question 1 [4 marks]

Write the function **bool search(Node* head, int key)** that searches for a given integer key in the linked list. (Do **not** modify the linked list.)

- If the key is found in any node, return true, otherwise, return false.

<pre>//Consider the following node structure struct Node { int data; Node* next; };</pre>	<pre>bool search(Node* head, int key) { Node* temp = head; while (temp != nullptr) { if (temp->data == key) { return true; } temp = temp->next; } return false; }</pre>
--	---

Question 2 [3 marks]

You are given the **Node** structure (given above) and the function **void tricky(Node* head)**. Dry run the following linked list

34 → 20 → 12 → 65 → 79 → 34 → NULL

Write down the output produced by the function

<pre>void tricky(Node* head) { Node* p = head; Node* q = head->next; while (q != NULL && q->next != NULL) { cout << p->data << " "; p = q->next; q = p->next; } }</pre>	Output: 34 12
--	------------------------------------

Question 3 [8 marks]

Write a function **Node* mergeTwoLists(Node* list1, Node* list2);**

- The function should take the **heads of two sorted singly linked lists** and return the head of a new **merged linked list** that is also sorted.
- You must perform the merge **in-place** (by rearranging the given nodes, without creating new ones).
- The merged list can contain duplicates

Example:

List1: 1 → 3 → 5 → NULL

List2: 2 → 4 → 6 → NULL

Merged List: 1 → 2 → 3 → 4 → 5 → 6 → NULL

```
Node* mergeTwoLists(Node* list1, Node* list2) {
    if (list1 == NULL) return list2;
    if (list2 == NULL) return list1;

    Node* head = NULL; // new head of merged list
    Node* tail = NULL; // tail pointer

    // Initialize head with smaller value
    if (list1->data <= list2->data) {
        head = list1;
        list1 = list1->next;
    } else {
        head = list2;
        list2 = list2->next;
    }
    tail = head; // tail starts at head

    // Traverse both lists
    while (list1 != NULL && list2 != NULL) {
        if (list1->data <= list2->data) {
            tail->next = list1;
            list1 = list1->next;
        } else {
            tail->next = list2;
            list2 = list2->next;
        }
        tail = tail->next; // move tail
    }

    // Attach remaining nodes
    if (list1 != NULL) tail->next = list1;
    else tail->next = list2;

    return head;}
```